PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Full Stack Authentication System

Student / Team Information

Team Name:	*Code_clash*
Team member 1 (Team Lead) *Name : Anshika Saklani* *University Roll no : 2318420* *Student iD : 23012076* *Email: anshikasaklani894@gmail.com*	
Team member 2 *Name : Ayush Chand* *University Roll no : 2318582* *Student iD : 230112536* *Email: ayushchand862@gmail.com*	

Team member 3 *Name : Akriti Rawat* *University Roll no : 2318263* *Student iD : 230221541* *Email: akritirawat12345@gmail.com*	
Team member 4 *Name : Rakhi Bisht* *University Roll no : 2319380* *Student iD : 23012177* *Email: rakhibisht24122004@gmail.com*	

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

The Problem: Vulnerability in Single-Factor Authentication
Traditional password-based security is no longer sufficient. With the rise of automated credential stuffing and sophisticated phishing, a single password represents a ***critical point of failure****. Once a password is leaked or guessed, an attacker gains full access to sensitive user data, leading to financial loss, identity theft, or corporate espionage.*

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Our project addresses the inherent weakness of Single-Factor Authentication (SFA) by building a ***Full-Stack Multi-Layered Security System****. The primary problem we are solving is the lack of verified identity during the login process. By integrating* ***Two-Factor Authentication (2FA)*** *via Google Authenticator, we ensure that even a compromised password is useless without the time-sensitive* ***TOTP (Time-based One-Time Password)*** *generated on the user's physical device.*

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The primary goal of this project is to develop a ***hardened, production-ready authentication framework*** *that eliminates the "single point of failure" inherent in traditional password systems. By merging modern cryptography with a scalable architecture, we aim to provide a secure environment for sensitive data handling in enterprise and financial applications.*

Core Objectives

- ***Zero-Trust Identity Verification:*** *Implement multi-factor layers (TOTP) to ensure identity is verified beyond a simple password.*
- ***Scalable Architecture:*** *Use stateless JWTs to support horizontal scaling without server-side session overhead.*
- ***Granular Access Control:*** *Define clear boundaries between standard users and administrative oversight via RBAC.*

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

*Our approach follows a **decoupled, modular architecture** centered on the principle of "Defense in Depth." By separating the identity provider logic from the data layer, we ensure the system is both secure and highly maintainable.*

Technical Design Strategy

*The solution is built on a **RESTful API architecture** using **Node.js and Express.js**. We utilize a stateless authentication flow where the server issues a **JSON Web Token (JWT)** upon successful credential verification. To prevent unauthorized access even if a token is intercepted, we implement **Two-Factor Authentication (2FA)**. This requires the backend to validate a Time-based One-Time Password (TOTP) generated via the **Google Authenticator** algorithm before granting a session.*

Technology Stack

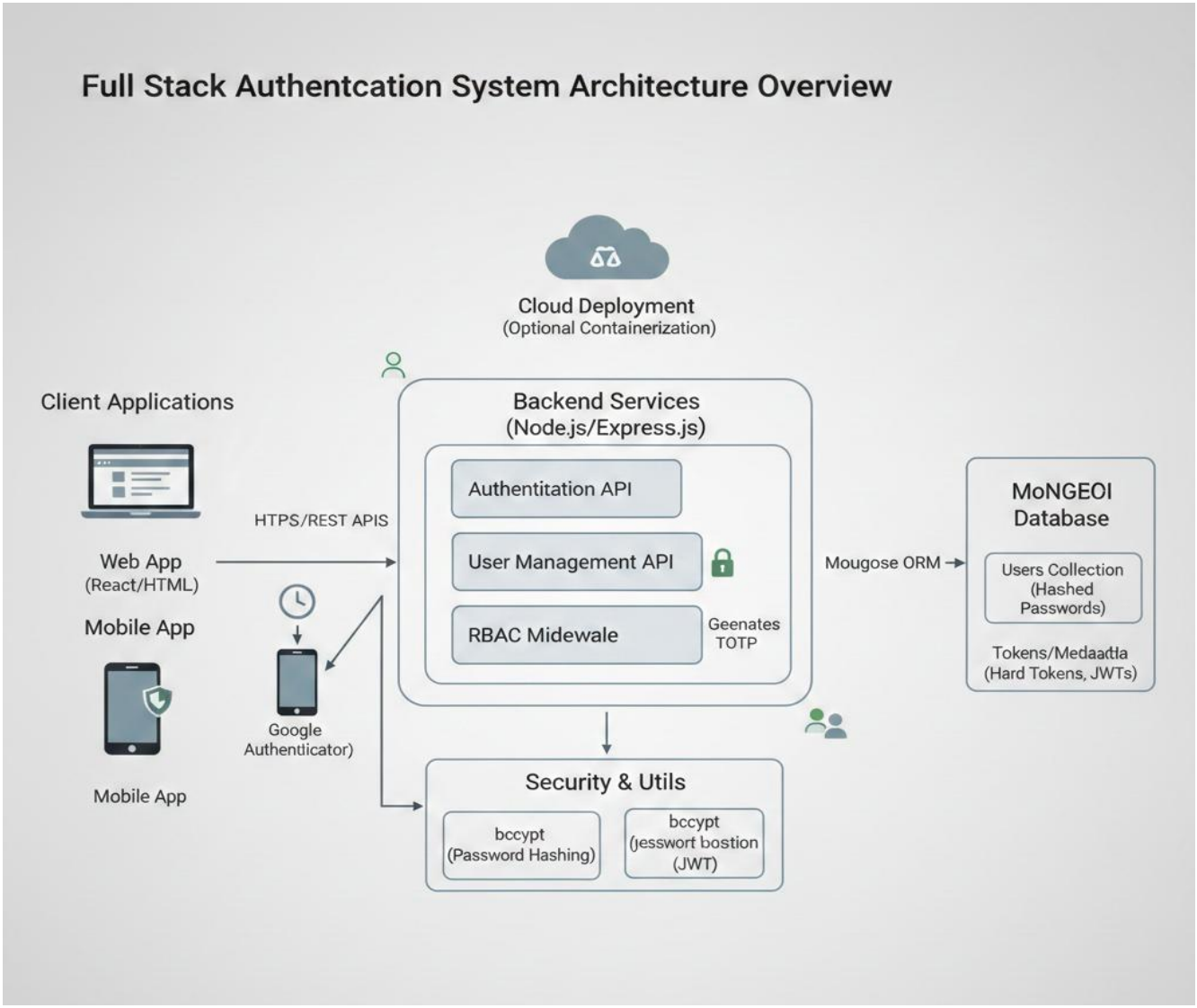
- ***Backend: Node.js** for its non-blocking I/O, paired with **Express.js** to manage routing and middleware-based security checks.*
- ***Database: MongoDB** with **Mongoose** modeling. This NoSQL approach allows for flexible storage of user profiles, hashed secrets, and administrative hard tokens.*
- ***Security Libraries:***
  - ***bcrypt:** For salted, multi-round password hashing.*
  - ***jsonwebtoken:** For generating and verifying signed access tokens.*
  - ***speakeasy / otplib:** To handle the TOTP logic for Google Authenticator integration.*
- ***Frontend:** A responsive interface built with **HTML/CSS** (and optionally **React.js**) to provide a seamless user experience for 2FA enrollment and admin management.*
- ***DevOps & Testing: Postman** for rigorous API testing and **dotenv** for secure environment variable management (API keys, DB URIs).*

Data Integrity & RBAC

*The design incorporates **Role-Based Access Control (RBAC)** middleware. Every request is intercepted to verify the user's permissions, ensuring that only authenticated Admins can access the hard token reset functions. This granular control protects the system from internal privilege escalation.*

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

*The final delivery of this project is a **hardened, production-ready authentication framework** designed for high-security enterprise environments.*

Core Deliverables

- ***Security-First Backend:*** *A Node.js/Express API featuring **bcrypt** password hashing and stateless **JWT** session management.*
- ***2FA Integration:*** *A functional module for **Google Authenticator**, including QR code generation for user enrollment and TOTP validation logic.*
- ***Admin Dashboard:*** *A management interface for **Role-Based Access Control (RBAC)** and manual administrative oversight.*

- ***Recovery System:*** *A secure* ***Hard Token-based*** *password reset mechanism, replacing vulnerable email-only workflows.*

Technical Assets

- ***API Documentation:*** *Comprehensive endpoint specifications, request/response schemas, and error codes.*
- ***Database Schema:*** *A documented* ***MongoDB/Mongoose*** *model optimized for storing encrypted credentials and role metadata.*
- ***Deployment Blueprint:*** *Documentation for containerization and environment configuration for horizontal scaling.*

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

To ensure a successful implementation, we are working under the following core assumptions:

- ***User Device Availability:*** *Users have access to a smartphone capable of running the* ***Google Authenticator*** *app (or a compatible TOTP client) for 2FA enrollment.*
- ***Persistent Connectivity:*** *The client applications maintain a stable internet connection to communicate with the* ***Node.js REST APIs*** *for real-time token validation.*
- ***Secure Environment Management:*** *Sensitive keys, such as* ***JWT secrets*** *and* ***MongoDB URIs****, will be stored in secure environment variables (.env) and not hard-coded.*
- ***Administrative Integrity:*** *Admin accounts are managed by trusted personnel, as they hold high-level permissions for* ***RBAC*** *and* ***Hard Token*** *resets.*

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

Core Documentation

- ***Node.js Official Documentation:*** *For implementing asynchronous backend logic and RESTful API structures. nodejs.org*
- ***MongoDB & Mongoose:*** *Used for data modeling, secure credential storage, and schema validation. mongodb.com/docs | mongoosejs.com*
- ***Express.js Guide:*** *Referenced for middleware implementation, specifically for* ***RBAC*** *and session handling. expressjs.com*

Security & Standards

- ***OWASP Authentication Cheat Sheet:*** *Guidance on industry-standard password hashing (bcrypt), salting, and session security. owasp.org*
- ***NIST Digital Identity Guidelines (SP 800-63B):*** *Followed for multi-factor authentication (MFA) requirements and TOTP (Time-based One-Time Password) implementation logic. nist.gov*
- ***RFC 6238 (TOTP):*** *The technical standard utilized for ensuring compatibility with the* ***Google Authenticator*** *app. ietf.org/rfc/rfc6238.txt*

Libraries & Tools

- ***bcrypt.js:*** *Documentation for secure, adaptive password hashing. npmjs.com/package/bcrypt*
- ***jsonwebtoken (JWT):*** *Best practices for stateless authentication and token signing. jwt.io*
- ***otplib / speakeasy:*** *Core libraries used to generate and verify Google Authenticator-compatible secrets. npmjs.com/package/otplib*